法律声明

- 本课件包括演示文稿、示例、代码、题库、视频和声音等内容，小象学院和主讲老师拥有完全知识产权的权利；只限于善意学习者在本课程使用，不得在课程范围外向任何第三方散播。任何其他人或机构不得盗版、复制、仿造其中的创意及内容，我们保留一切通过法律手段追究违反者的权利。

- 课程详情请咨询
  - 微信公众号：小象
  - 新浪微博：ChinaHadoop

贪心法和动态规划

小象学院 ChinaHadoop.cn

邹博

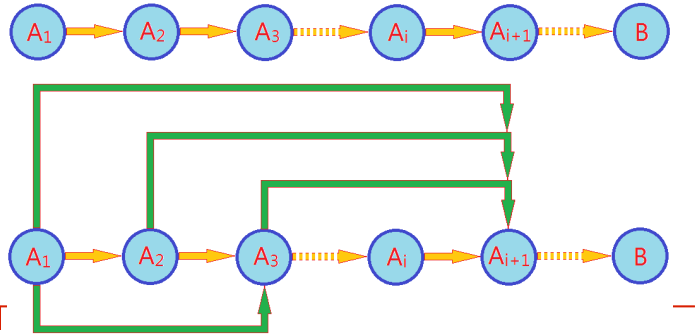
主要内容

- 动态规划和贪心的认识
  - 工具：马尔科夫过程
- 贪心法
  - Prim算法/Kruskal算法
  - Dijkstra算法→(Floyd算法：贪心/动态规划？)
- 动态规划
  - 最长递增子序列LIS
  - 矩阵连乘的最少次数
  - 找零钱/背包问题
  - 两次走棋盘
  - 括号有效字符串问题：Catalan数

认识论

- 认识事物的方法：概念、判断、推理
- 推理中，又分为归纳、演绎。
- 重点考察归纳推理的具体方法。
- 形式化表述：
  - 已知：问题规模为n的前提A
  - 求解/求证：未知解B/结论B
  - 记号：用A_n表示“问题规模为n的已知条件”

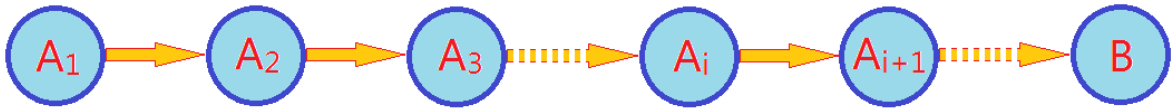
对归纳推理的理解



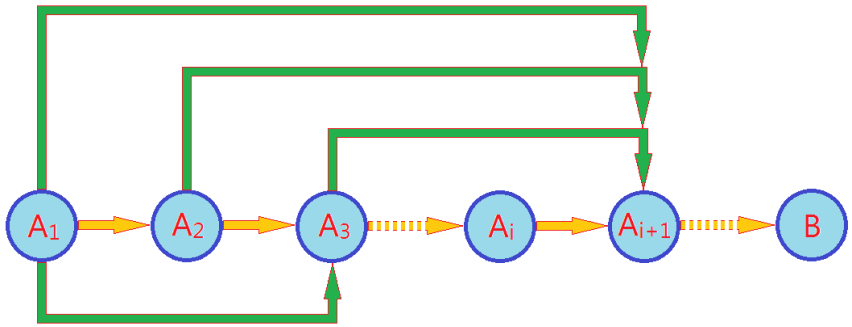
- 若将问题规模降低到0，即已知A_0，很容易计算或证明B，则有：$A_0 \rightarrow B$
- 同时，考察从A_0增加一个元素，得到A1的变化过程。即：$A_0 \rightarrow A_1$；
  - 进一步考察$A_1 \rightarrow A_2$，$A_2 \rightarrow A_3$ ……$A_i \rightarrow A_{i+1}$
  - 这种方法是(严格的)归纳推理，常常被称作**数学归纳法**。
  - 此时，由于上述推导往往不是等价推导(A_i和A_{i+1}不是互为充要条件)，导致随着i的增加，有价值的前提信息越来越少；为避免这一问题，采取如下方案：
  - $\{A_1\} \rightarrow A_2$，$\{A_1A_2\} \rightarrow A_3$ …… $\{A_1A_2 \ldots A_i\} \rightarrow A_{i+1}$
  - 相对应的，修正后的方法依然是(严格的)归纳推理，有时被称作**第二数学归纳法**。

对归纳推理的理解

- 基本归纳法：对于A_{i+1}，只需考察前一个状态A_i即可完成整个推理过程，它的特点是只要状态A_i确定，则计算A_{i+1}时不需要考察更前序的状态$A_1 \ldots A_{i-1}$，在图论中，常常称之为马尔科夫模型；

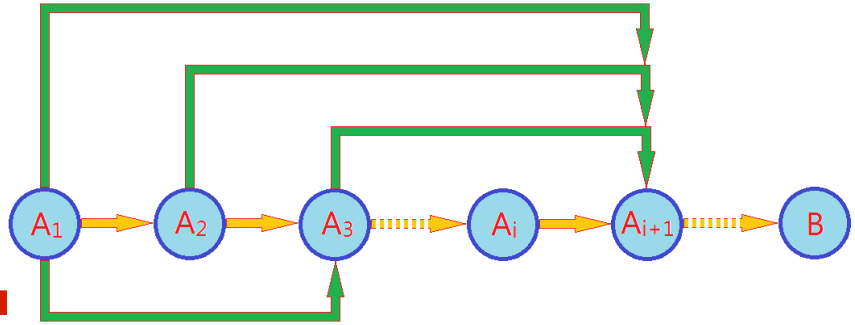


- 高阶归纳法：相应的，对于A_{i+1}，需考察前i个状态集$\{A_1A_2 \ldots A_{i-1}A_i\}$才可完成整个推理过程，往往称之为高阶马尔科夫模型；



- 在计算机算法中，高阶马尔科夫模型的推理，叫做“动态规划”，而马尔科夫模型的推理，对应“贪心法”。

以上理解的说明



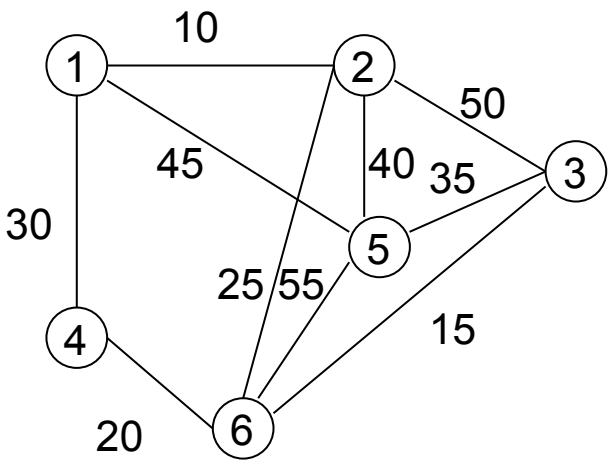
- 无论动态规划还是贪心法，都是根据A[0…i]计算A[i+1]的过程
  - 计算A[i+1]不需要A[i+2]、A[i+3]……，
  - 一旦计算完成A[i+1]，再后面计算A[i+2]、A[i+3]……时，不会更改A[i+1]的值。
  - 这即无后效性。
- 亦可以如下理解动态规划：计算A[i+1]只需要知道A[0…i]的值，无需知道A[0…i]是通过何种途径计算得到的——只需知道它们当前的状态值本身即可。如果将A[0…i]的全体作为一个整体，则可以认为动态规划法是马尔科夫过程，而非高阶马尔科夫过程。

最小生成树MST

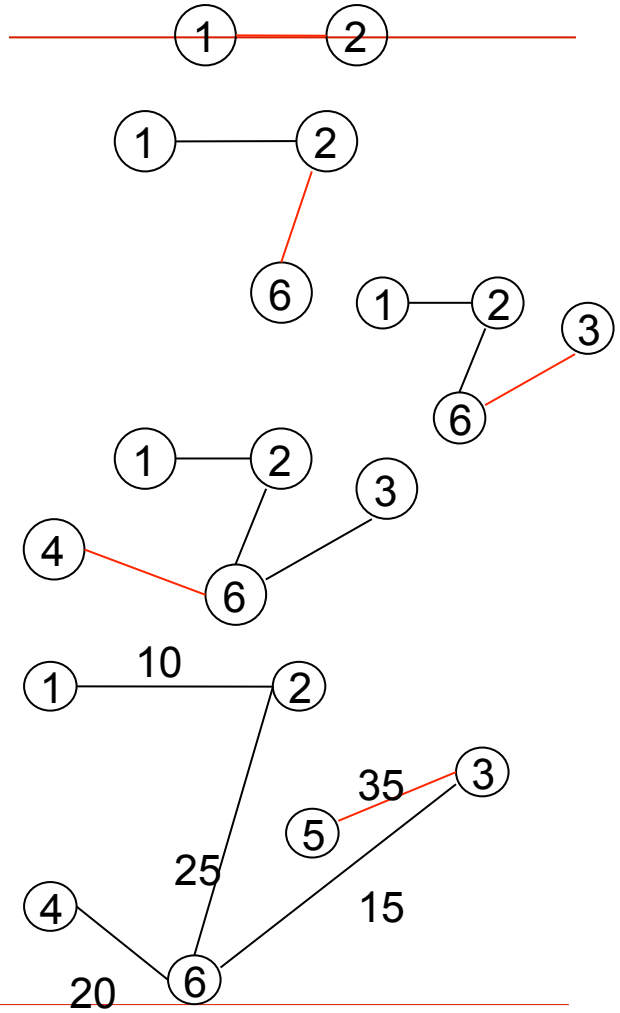
- 最小生成树要求从一个带权无向连通图中选择n－1条边并使这个图仍然连通(也即得到了一棵生成树)，同时还要考虑使树的权最小。最著名的是Prim算法和Kruskal算法，它们都是贪心算法。
  - Prim算法：从某个(任意一个)结点出发，选择与该结点邻接的权值最小的边；随着结点的不断加入，每次都选择这些结点发出的边中权值最小的：重复n-1次。
  - Kruskal算法：将边按照权值递增排序，每次选择权值最小并且不构成环的边，重复n-1次。

Prim算法

使得所选择的边的集合A构成一棵树；将要计入到A中的下一条边(u,v)，应是E中一条当前不在A中且使得A∪{(u,v)}是一棵树的最小成本边。

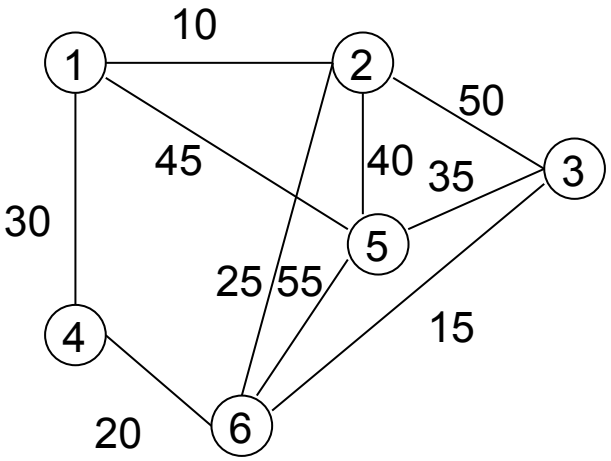


边	成本
(1,2)	10
(2,6)	25
(3,6)	15
(6,4)	20
(3,5)	35

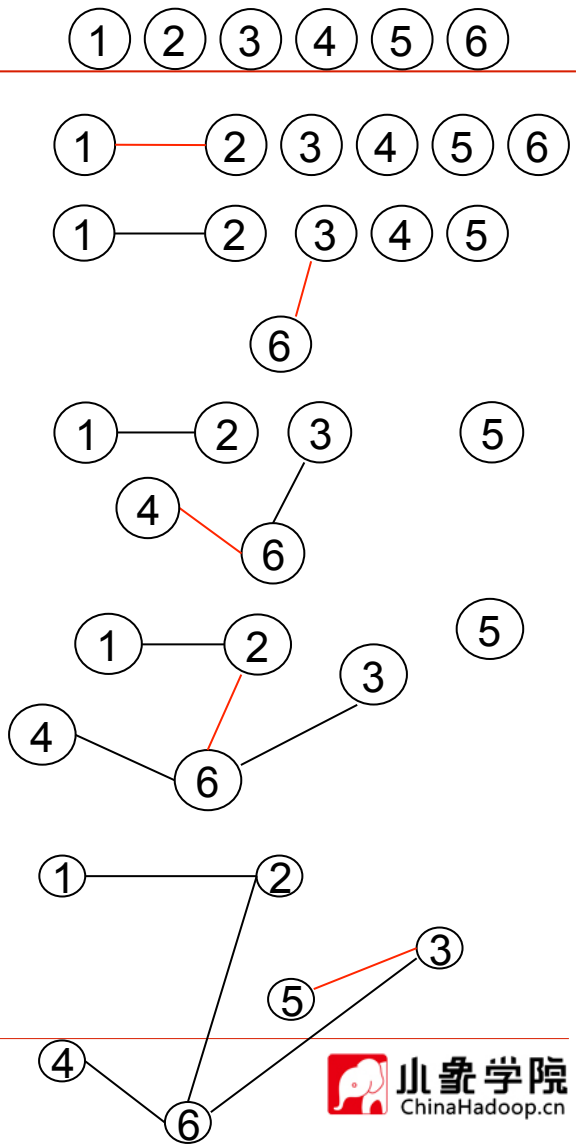


Kruskal算法

图G的所有边按成本非降次序排列，下一条生成树T中的边是尚未加入树的边中具有最小成本、且和T中现有的边不会构成环路的边。



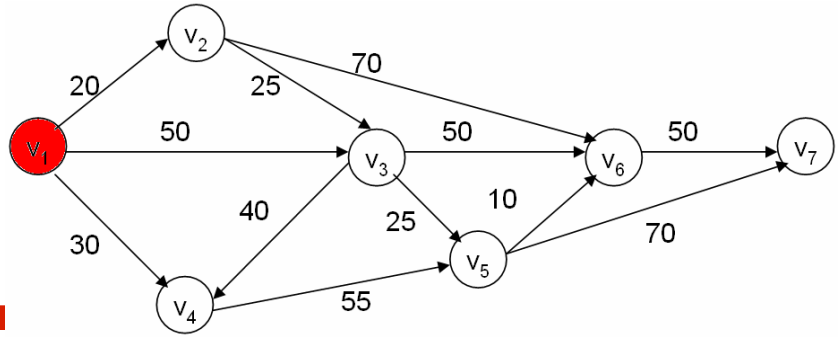
边	成本
(1,2)	10
(3,6)	15
(4,6)	20
(2,6)	25
(3, 5)	35



Dijkstra最短路径

- 将Prim算法稍做调整，得到Dijkstra算法：
  - 初始化结点集S：S={v_0}，初始化到每个点的最短路径距离dist[u]=graph[v_0][u]；
  - 选择最小的dist[u]：记dist[v]最小，则v是不在S中且距离v_0最近的结点，更新S=S∪{v}，调整dist[u]=min{dist[u],dist[v]+graph[v][u]}；
  - 重复n-1次。

Dijkstra最短路径



迭代	选取的结点	S	DIST (1)	(2)	(3)	(4)	(5)	(6)	(7)
置初值	—	1	0	20	50	30	+∞	+∞	+∞
1	2	1,2	0	20	45	30	+∞	90	+∞
2	4	1,2,4	0	20	45	30	85	90	+∞
3	3	1,2,4,3	0	20	45	30	70	90	+∞
4	5	1,2,4,3,5	0	20	45	30	70	90	140
5	6	1,2,4,3,5,6	0	20	45	30	70	90	130

- 有n-1个结点加入到S后终止，此时dist[0…n-1]中记录了v_0至其它各结点的最短路径的长度。
- 问题：如何求出所有最短路径本身？
  - 记录前驱

贪心法的思考

- 可以看到，在从A_i到A_{i+1}的扩展过程中，上述Prim/Kruskal/Dijkstra三个算法都没有使用A[0…i-1]的值。
- 人们往往根据名字“贪心法”认为它很简单；事实上，贪心法其实并不轻松，它需要严格证明一定与更先序的值无关。
  - 思考：从1元、2元、5元的纸币，问给定总价值N元，最少需要几张纸币？

Floyd算法

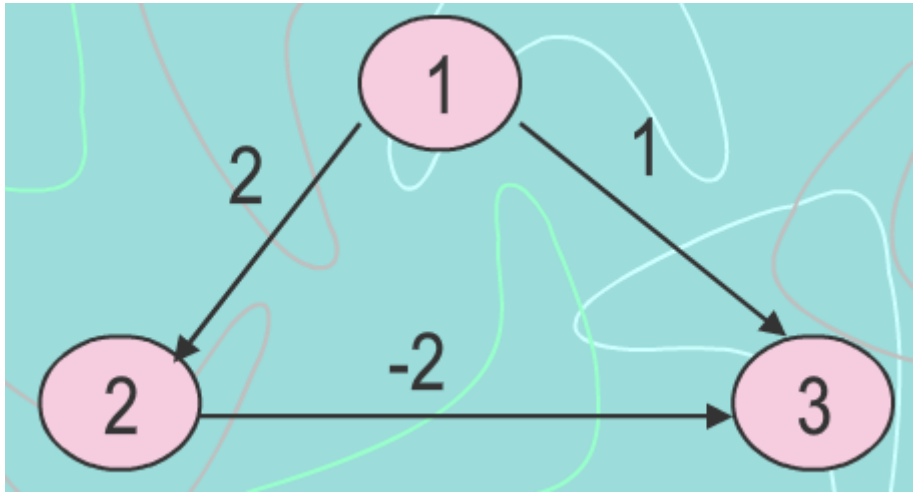
- Floyd算法又称插点法，是一种用于寻找给定的加权图中多源点之间最短路径的算法。
- 记map[i,j]为结点i到结点j的最短路径的距离；则：
- map[i,j] = min{map[i,k] + map[k,j], map[i,j]}
  - k取所有结点
- map[n,n]==0
- i,j,k各自从0到N-1，所以时间复杂度为$O(n^3)$
- 如果图中存在负的权值，算法也是适用的。
  - 思考：Dijkstra算法允许边存在负权吗？

Floyd算法

```
D[u,v]=A[u,v]   //初始化
For k:=1 to n
  For i:=1 to n
    For j:=1 to n
      If D[i,j]>D[i,k]+D[k,j] Then
        D[I,j]:=D[I,k]+D[k,j];
```

附：带负权的最短路径

- 计算图中最小的权值，若该权值大于0，按照Dijkstra算法正常计算；若小于0，则所有权值都加上该权值的绝对值+1，则修改后的图不再含有负权。使用Dijkstra算法计算最小路径。
- 是否可行？



附：Bellman-ford算法

- 本质：动态规划
- 适用：单源结点到其他所有结点的最短路径
- 若u→v是有向边，则$d[v] \le d[u] + dis(u,v)$
- 这个操作被成为松弛操作。
- 优点：对边权无要求，可以发现负环。

附： Code

```
//邻接矩阵为graph，结点数目为N，计算start到其他所有点的最短路径
void BellmanFord(int graph[N][N], int N, int* path, int start)
{
    int i,j,k;
    for(i = 0; i < N; i++)
        path[i] = 1000000;

    path[start] = 0;
    for(i = 0; i < N; i++)
    {
        //对于每条边
        for(j = 0; j < N; j++)
        {
            for(k = 0; k < N; k++)
            {
                if(graph[j][k] != 0)    //说明有边
                {
                    path[k] = min(path[k], path[j] + graph[j][k]);
                }
            }
        }
    }
}
```

Code

```
//邻接矩阵为graph，结点数目为N，计算start到其他所有点的最短路径
void SPFA(int graph[N][N], int N, int* path, int start)
{
    int i;
    for(i = 0; i < N; i++)
        path[i] = 1000000;
    path[start] = 0;

    queue<int> q;
    q.push(0);
    int t;
    int count = 0;
    while(!q.empty())
    {
        t = q.front();
        q.pop();
        for(i = 0; i < N; i++)
        {
            if(graph[t][i] != 0)
            {
                if(path[i] > path[t] + graph[t][i]) //经过t的新路径能够更短
                {
                    path[i] = path[t] + graph[t][i];
                    q.push(i);
                }
            }
        }
    }
}
```

最长递增子序列LIS

- Longest Increasing Subsequence
- 给定长度为N的数组A，计算A的最长的单调递增的子序列(不一定连续)。
  - 如：给定数组A{5,6,7,1,2,8}，则A的LIS为{5,6,7,8}，长度为4。

附：使用LCS解LIS问题

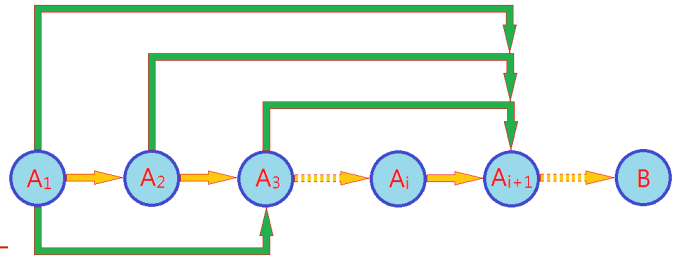
- 原数组为A {5， 6， 7， 1， 2， 8}
- 排序后：A’ {1， 2， 5， 6， 7， 8}
- 因为，原数组A的子序列顺序保持不变，而且排序后A’本身就是递增的，这样，就保证了两序列的最长公共子序列的递增特性。如此，若想求数组A的最长递增子序列，其实就是求数组A与它的排序数组A’的最长公共子序列。

前缀分析

Array	1	4	6	2	8	9	7
LIS	1	2	3	2	4	5	4

- “1”：1
- “14”：14
- “146”：146
- “1462”：146
- “14628”：1468
- “146289”：14689
- “1462897”：14689

最长递增子序列LIS记号



- 长度为N的数组记为$A=\{a_0a_1a_2...a_{n-1}\}$；
- 记A的前i个字符构成的前缀串为$A_i=a_0a_1a_2...a_{i-1}$，以a_i结尾的最长递增子序列记做L_i，其长度记为b[i]；
- 假定已经计算得到了b[0,1…,i-1]，如何计算b[i]呢？
  - 已知L_0 L_1…… L_{i-1}的前提下，如何求L_i？

求解LIS

Array	1	4	6	2	8	9	7
LIS	1	2	3	2	4	5	4

- 根据定义，L_i必须以a_i结尾；
- 如果将a_i分别缀到L_0 L_1…… L_{i-1}后面，是否允许呢？
  - 如果$a_i \geq a_j$，则可以将a_i缀到L_j的后面，得到比L_j更长的字符串。
- 从而：$b[i]=\{\max(b[j])+1, 0 \leq j < i$且$a_j \leq a_i\}$
  - 计算b[i]：遍历在i之前的所有位置j，找出满足条件$a_j \leq a_i$的最大的b[j]+1；
  - 计算得到b[0…n-1]后，遍历所有的b[i]，找出最大值即为最大递增子序列的长度。
- 时间复杂度为$O(N^2)$。

LIS的思考

- 思考：如何求最大递增子序列本身？
  - 记录前驱

Array	1	4	6	2	8	9	7
LIS	1	2	3	2	4	5	4

Code1

```
int LIS1(const int* p, int length)
{
    int* longest = new int[length];
    int i,j;

    for(i = 0; i < length; i++)
        longest[i] = 1;

    int nLis = 1;
    for(i = 1; i < length; i++)
    {
        for(j = 0; j < i; j++)
        {
            if(p[j] <= p[i])
            {
                longest[i] = max(longest[i], longest[j]+1);
            }
        }
        nLis = max(nLis, longest[i]);
    }
    delete[] longest;
    return nLis;
}
```

LIS Code split

```
int _tmain(int argc, _TCHAR* argv[])
{
    int array[] = {1,4,5,6,2,3,8,9,10,11,12,12,1};
    int size = sizeof(array)/sizeof(int);
    int* pre = new int[size];
    int nIndex;
    int max = LIS(array, size, pre, nIndex);
    vector<int> lis;
    GetLIS(array, pre, nIndex, lis);
    delete[] pre;
    cout << max << endl;
    Print(&lis.front(), (int)lis.size());
    return 0;
}
```

```
void GetLIS(const int* array, const int* pre,
            int nIndex, vector<int>& lis)
{
    while(nIndex >= 0)
    {
        lis.push_back(array[nIndex]);
        nIndex = pre[nIndex];
    }
    reverse(lis.begin(), lis.end());
}
```

```
#include <vector>
#include <algorithm>
using namespace std;

int LIS(const int* p, int length, int* pre, int& nIndex)
{
    int* longest = new int[length];
    int i,j;

    for(i = 0; i < length; i++)
    {
        longest[i] = 1;
        pre[i] = -1;
    }

    int nLis = 1;
    nIndex = 0;
    for(i = 1; i < length; i++)
    {
        for(j = 0; j < i; j++)
        {
            if(p[j] <= p[i])
            {
                if(longest[i] < longest[j]+1)
                {
                    longest[i] = longest[j]+1;
                    pre[i] = j;
                }
            }
        }
        if(nLis < longest[i])
        {
            nLis = longest[i];
            nIndex = i;
        }
    }

    delete[] longest;

    return nLis;
}
```

```
void Print(int* p, int size)
{
    for(int i = 0; i < size; i++)
        cout << p[i] << '\t';
    cout << '\n';
}
```

O(NlogN)的最长递增子序列算法

- 对于数组A={1,4,6,2,8,9,7}
- 1
- 1,4
- 1,4,6
- 1,2,6
- 1,2,6,8
- 1,2,6,8,9
- 1,2,6,7,9

Code2

```
int LIS(const int* a, int size)
{
    int* b = new int[size];
    int s = 0;
    int i;
    for(i = 0; i < size; i++)
        Insert(b, s, a[i]);
    delete[] b;
    return s;
}

int _tmain(int argc, _TCHAR* argv[])
{
    int a[] = {5,6,7,1,2,8};
    int size = sizeof(a) / sizeof(int);
    int len = LIS(a, size);
    cout << len << endl;
    return 0;
}
```

```
void Insert(int* a, int& size, int x)
{
    if(size <= 0)
    {
        a[0] = x;
        size++;
        return;
    }
    int low = 0;
    int high = size-1;
    int mid;
    while(low <= high)
    {
        mid = (low+high)/2;
        if(x < a[mid])
            high = mid-1;
        else if(x >= a[mid])
            low = mid+1;
    }

    if(low >= size)
    {
        a[size] = x;
        size++;
    }
    else
    {
        if(a[low] < x)
            a[low+1] = x;
        else
            a[low] = x;
    }
}
```

矩阵连乘的提法

- 给定n个矩阵$\{A_1,A_2,\ldots,A_n\}$，其中A_i与A_{i+1}是可乘的，i=1，2…，n-1。考察该n个矩阵的连乘积：$A_1\times A_2\times A_3\ldots\ldots\times A_n$，确定计算矩阵连乘积的计算次序，使得依此次序计算矩阵连乘积需要的乘法次数最少。
  - 即：利用结合律，通过加括号的方式，改变计算过程，使得数乘的次数最少。

分析

- 将矩阵连乘积$A_i A_{i+1} \ldots A_j$ 记为A[i:j]，这里i≤j
  - 显然，若i==j，则A[i:j]即A[i]本身。
- 考察计算A[i:j]的最优计算次序。设这个计算次序在矩阵A_k和A_{k+1}之间将矩阵链断开，i≤k<j，则其相应的完全加括号方式为

$$(A_i A_{i+1} \ldots A_k)(A_{k+1} A_{k+2} \ldots A_j)$$

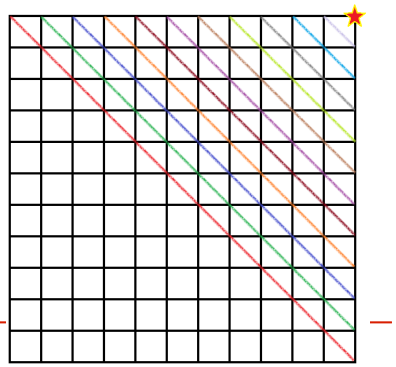
- 计算量：A[i:k]的计算量加上A[k+1:j]的计算量，再加上A[i:k]和A[k+1:j]相乘的计算量

状态转移方程 $(A_i A_{i+1} \ldots A_k)(A_{k+1} A_{k+2} \ldots A_j)$

- 设计算A[i:j](1≤i≤j≤n)所需要的最少数乘次数为m[i,j]，则原问题的最优值为m[1,n]；
- 记A_i的维度为 $p_{i-1} \times p_i$
- 当i==j时，A[i:j]即A_i本身，因此，m[i,i]=0；(i=1,2,…,n)
- 当i<j时，$m[i,j] = m[i,k] + m[k+1,j] + p_{i-1} p_k p_j$
- 从而：

$$m[i,j] = \begin{cases} 0 & i = j \\ \min\limits_{i \le k < j} \{m[i,k] + m[k+1,j] + p_{i-1} p_k p_j\} & i < j \end{cases}$$

矩阵连乘问题从算法到实现

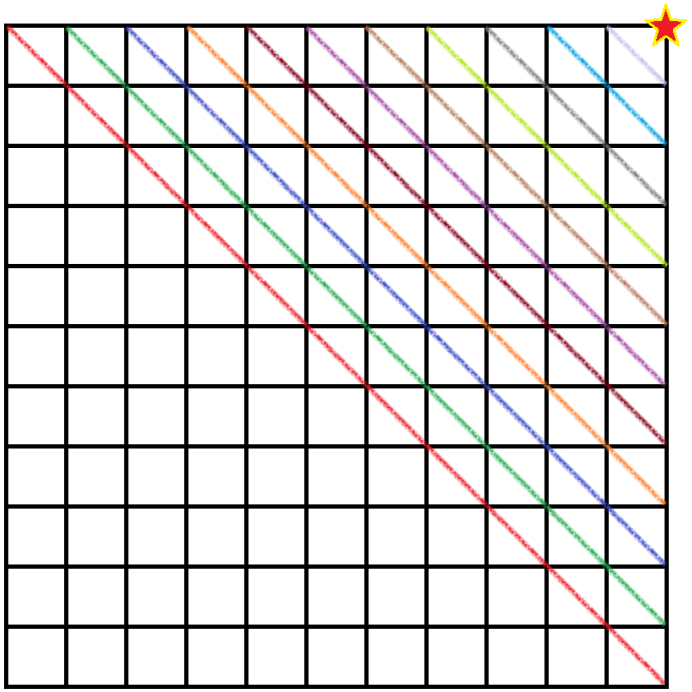


- 由m[i,j]的递推关系式可以看出，在计算m[i,j]时，需要用到m[i+1,j]，m[i+2,j]…m[j-1,j]；

$$m[i,j]=\begin{cases}0 & i=j\\ \min\limits_{i\le k<j}\{m[i,k]+m[k+1,j]+p_{i-1}p_kp_j\} & i<j\end{cases}$$

- 因此，求m[i,j]的前提，不是m[0…i-1;0…j-1]，而是沿着主对角线开始，依次求取到右上角元素。
- 因为m[i,j]一个元素的计算，最多需要遍历n-1次，共$O(n^2)$个元素，故算法的时间复杂度是$O(n^3)$，空间复杂度是$O(n^2)$。

Code



```
//p[0...n]存储了n+1个数，其中，(p[i-1],p[i])是矩阵i的阶；
//s[i][j]记录A[i...j]从什么位置断开；m[i][j]记录数乘最小值
void MatrixMultiply(int* p, int n, int** m, int** s)
{
    int r,i,j,k,t;
    for(i = 1; i <= n; i++)
        m[i][i] = 0;

    //r个连续矩阵的连乘：上面的初始化，相当于r=1
    for(r = 2; r <= n; r++)
    {
        for(i = 1; i <= n-r+1; i++)
        {
            j=i+r-1;
            m[i][j] = m[i+1][j]+ p[i-1]*p[i]*p[j];
            s[i][j] = i;
            for(k = i+1; k < j; k++)
            {
                t = m[i][k] + m[k+1][j] + p[i-1]*p[k]*p[j];
                if(t < m[i][j])
                {
                    m[i][j] = t;
                    s[i][j] = k;
                }
            }
        }
    }
}
```

找零钱

- 给定某不超过100万元的现金总额，兑换成数量不限的100、50、20、10、5、2、1的组合，共有多少种组合呢？

该问题的思考过程

- 此问题涉及两个类别：面额和总额。
  - 如果面额都是1元的，则无论总额多少，可行的组合数显然都为1。
  - 如果面额多一种，则组合数有什么变化呢？
- 定义dp[i][j]：使用面额小于等于i的钱币，凑成j元钱，共有多少种组合方法。
  - dp[100][500] = dp[50][500] + dp[100][400]
  - dp[i][j] = dp[i_{small}][j] + dp[i][j-i]
    - 不考虑*j-i*下溢出等边界问题

递推公式 $dp[i][j] = dp[i_{small}][j] + dp[i][j-i]$

- 使用dom[]={1,2,5,10,20,50,100}表示基本面额，i的意义从面额变成面额下标，则：
  - dp[i][j] = dp[i-1][j] + dp[i][j-dom[i]]
- 从而：

$$dp[i][j] = \begin{cases} dp[i-1][j] + dp[i][j-dom[i]], & j \geq dom[i] \\ dp[i-1][j], & j < dom[i] \end{cases}$$

- 初始条件：

$$\begin{cases} dp[0][j] = 1 \\ dp[i][0] = 1 \end{cases}$$

Code

```
int Charge(int value, const int* denomination, int size)
{
    int i;
    int** dp = new int*[size]; //dp[i][j]: 用i面额以下的组合成j元
    for(i = 0; i < size; i++)
        dp[i] = new int[value+1];

    int j;
    for(j = 0; j <= value; j++) //只用面额1元的
        dp[0][j] = 1;

    for(i = 1; i < size; i++)  //先用面额小的，再用面额大的
    {
        dp[i][0] = 1;  //原因：添加任何一个面额，就是一个有效组合
        for(j = 1; j <= value; j++)
        {
            if(j >= denomination[i])
                dp[i][j] = dp[i-1][j] + dp[i][j-denomination[i]];
            else
                dp[i][j] = dp[i-1][j];
        }
    }
    int time = dp[size-1][value];

    for(i = 0; i < size; i++)  //清理内存
        delete[] dp[i];
    delete[] dp;
    return time;
}

int _tmain(int argc, _TCHAR* argv[])
{
    int denomination[] = {1,2,5,10,20,50,100}; //面额
    int size = sizeof(denomination) / sizeof(int);
    int value = 200;
    int c = Charge(value, denomination, size);
    cout << c << endl;
    return 0;
}
```

滚动数组

- 将状态转移方程去掉第一维，很容易使用滚动数组，降低空间使用量。
- 原状态转移方程：

$$dp[i][j]=\begin{cases} dp[i-1][j]+dp[i][j-dom[i]], & j\geq dom[i] \\ dp[i-1][j], & j<dom[i] \end{cases}$$

- 滚动数组版本的状态转移方程：

$$dp[j]=last[j]+dp[j-dom[i]],\quad (j\geq dom[i])$$

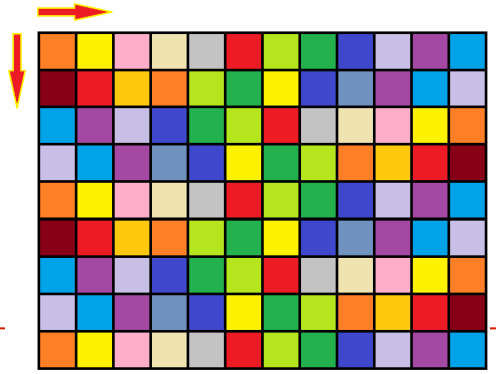
Code2

```
int Charge2(int value, const int* denomination, int size)
{
    int i;
    int* dp = new int[value+1]; //dp[j]: 凑成j元的组合数
    int* last = new int[value+1];

    int j;
    for(j = 0; j <= value; j++) //只用面额1元的
    {
        dp[j] = 1;
        last[j] = 1;
    }

    for(i = 1; i < size; i++)   //先用面额小的，再用面额大的
    {
        for(j = 1; j <= value; j++)
        {
            if(j >= denomination[i])
                dp[j] = last[j] + dp[j-denomination[i]];
        }
        memcpy(last, dp, sizeof(int)*(value+1));
    }
    int chargeTimes = dp[value];

    delete[] last;
    delete[] dp;
    return chargeTimes;
}
```

总结与思考

- 请问：本问题的时间复杂度是多少？
- 在动态规划问题中，如果不求具体解，而只计算解的数目，往往可以使用滚动数组的方式降低空间使用量(甚至空间复杂度)
  - 由于滚动数组减少了维度，甚至代码会更简单
- 思考0-1背包问题和格子取数问题。

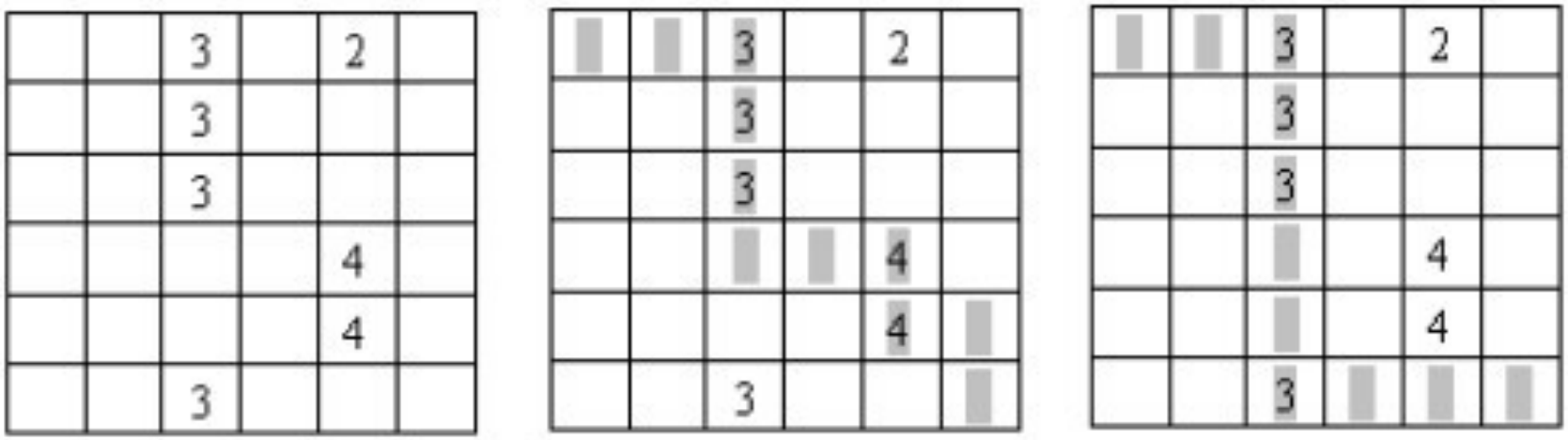
两次走棋盘



- 给定m×n的矩阵，每个格子为非负权值，从左上角开始，每次只能向右和向下走，直到右下角；然后，从右下角开始，每次只能向左和向上走，回到左上角。求经过格子的权值和最大的路径。
  - 若相同格子经过两次，则权值只算一次。

分析

贪心不能解决问题



图一　　图二　　图三

分析格网棋盘的特点

- 考察5×7的矩阵棋盘C，其中，C[i,j]表示的值v表示第v步能够到达的位置。

0	1	2	3	4	5	6
1	2	3	4	5	6	7
2	3	4	5	6	7	8
3	4	5	6	7	8	9
4	5	6	7	8	9	X

举例说明状态的定义

0	1	2	3	4	5	6
1	2	3	4	5	6	7
2	3	4	5	6	7	8
3	4	5	6	7	8	9
4	5	6	7	8	9	X

- 以经过8步为例：
- 在经过8步后，处于C中编号为8的位置。而C中共有3个编号为8的各自，分别是C的第2、3、4行。故假设第1次经过8步走到了C中的第2行，第2次经过8步走到了C中的第3行，用dp[8,2,3]表示。
- 用dp[s,i,j]记录两次所走的路径获得的最大值，其中s表示走的步数，i和j表示在s步后第1次走的位置和第2次走的位置。由于s=m+n-2，$0 \le i < n$，$0 \le j < m$，所以共有$O(n^3)$个状态。

状态转移函数的定义

0	1	2	3	4	5	6
1	2	3	4	5	6	$\boxed{7}$
2	3	4	5	6	$\boxed{7}$	$\boxed{8}$
3	4	5	6	$\boxed{7}$	$\boxed{8}$	9
4	5	6	7	8	9	X

状态转移函数的定义

0	1	2	3	4	5	6
1	2	3	4	5	6	$\boxed{7}$
2	3	4	5	6	$\boxed{7}$	$\boxed{8}$
3	4	5	6	7	8	9
4	5	6	7	8	9	X

完整的状态转移函数

- if(i!=j)
  - DP[s,i,j]=Max(
    - DP[s-1,i-1,j-1],
    - DP[s-1,i-1,j],
    - DP[s-1,i,j-1],
    - DP[s-1,i,j])
    - + W[i,s-i] + W[j,s-j]

0	1	2	3	4	5	6
1	2	3	4	5	6	7
2	3	4	5	6	7	8
3	4	5	6	7	8	9
4	5	6	7	8	9	X

- Else
  - DP[s,i,j]=Max(
    - DP[s-1,i-1,j-1],
    - DP[s-1,i-1,j],
    - DP[s-1,i,j])
    - +W[i,s-i]

0	1	2	3	4	5	6
1	2	3	4	5	6	7
2	3	4	5	6	7	8
3	4	5	6	7	8	9
4	5	6	7	8	9	X

- 其中W[x,y]表示棋盘位置(x,y)的权值。

Code

```
const int N = 202;
const int inf = 1000000000;  //无穷大
int dp[N * 2][N][N];
bool IsValid(int step, int x1, int x2, int n)
{
    int y1 = step - x1, y2 = step - x2;
    return ((x1 >= 0) && (x1 < n) && (x2 >= 0) && (x2 < n) && (y1 >= 0) && (y1 < n) && (y2 >= 0) && (y2 < n));
}

int GetValue(int step, int x1, int x2, int n)
{
    return IsValid(step, x1, x2, n) ? dp[step][x1][x2] : (-inf);
}

//dp[step][i][j]表示在第step步两次分别在第i行和第j行的最大得分
int MinPathSum(int a[N][N], int n)
{
    int P = n * 2 - 2; //最终的步数
    int i, j, step;

    //不能到达的位置 设置为负无穷大
    for(i = 0; i < n; ++i)
    {
        for(j = i; j < n; ++j)
        {
            dp[0][i][j] = -inf;
        }
    }
    dp[0][0][0] = a[0][0];

    for(step = 1; step <= P; ++step)
    {
        for(i = 0; i < n; ++i)
        {
            for(j = i; j < n; ++j)
            {
                dp[step][i][j] = -inf;
                if(!IsValid(step, i, j, n))    //非法位置
                    continue;
                //对于合法的位置进行dp
                if(i != j)
                {
                    dp[step][i][j] = max(dp[step][i][j], GetValue(step - 1, i - 1, j - 1, n));
                    dp[step][i][j] = max(dp[step][i][j], GetValue(step - 1, i - 1, j, n));
                    dp[step][i][j] = max(dp[step][i][j], GetValue(step - 1, i, j - 1, n));
                    dp[step][i][j] = max(dp[step][i][j], GetValue(step - 1, i, j, n));
                    dp[step][i][j] += a[i][step - i] + a[j][step - j];  //不在同一个格子，加两个数
                }
                else
                {
                    dp[step][i][j] = max(dp[step][i][j], GetValue(step - 1, i - 1, j - 1, n));
                    dp[step][i][j] = max(dp[step][i][j], GetValue(step - 1, i - 1, j,  n));
                    dp[step][i][j] = max(dp[step][i][j], GetValue(step - 1, i, j,  n));
                    dp[step][i][j] += a[i][step - i]; // 在同一个格子里，只能加一次
                }
            }
        }
    }
    return dp[P][n - 1][n - 1];
}
```

Code split

```
//dp[step][i][j]表示在第step步两次分别在第i行和第j行的最大得分
int MinPathSum(int a[N][N], int n)
{
    int P = n * 2 - 2; //最终的步数
    int i, j, step;

    //不能到达的位置 设置为负无穷大
    for(i = 0; i < n; ++i)
    {
        for(j = i; j < n; ++j)
        {
            dp[0][i][j] = -inf;
        }
    }
    dp[0][0][0] = a[0][0];

    for(step = 1; step <= P; ++step)
    {
        for(i = 0; i < n; ++i)
        {
            for(j = i; j < n; ++j)
            {
                dp[step][i][j] = -inf;
                if(!IsValid(step, i, j, n))    //非法位置
                    continue;
                //对于合法的位置进行dp
                if(i != j)
                {
                    dp[step][i][j] = max(dp[step][i][j], GetValue(step - 1, i - 1, j - 1, n));
                    dp[step][i][j] = max(dp[step][i][j], GetValue(step - 1, i - 1, j, n));
                    dp[step][i][j] = max(dp[step][i][j], GetValue(step - 1, i, j - 1, n));
                    dp[step][i][j] = max(dp[step][i][j], GetValue(step - 1, i, j, n));
                    dp[step][i][j] += a[i][step - i] + a[j][step - j];
                }
                else
                {
                    dp[step][i][j] = max(dp[step][i][j], GetValue(step - 1, i - 1, j - 1, n));
                    dp[step][i][j] = max(dp[step][i][j], GetValue(step - 1, i - 1, j,  n));
                    dp[step][i][j] = max(dp[step][i][j], GetValue(step - 1, i, j,  n));
                    dp[step][i][j] += a[i][step - i]; // 在同一个格子里，只能加一次
                }
            }
        }
    }
    return dp[P][n - 1][n - 1];
}
```

所有括号匹配的字符串

- N对括号能够得到的有效括号序列有哪些？
- 如N=3时，有效括号串共5个，分别为：
  - 1：()()()
  - 2：()(())
  - 3：(())()
  - 4：(()())
  - 5：((()))

问题分析

- 任何一个括号序列，都可以写成形式A(B)
  - A、B都是若干括号对形成的合法串(可为空串)
  - 若N=0，括号序列为空。
  - 若N=1，括号序列只能是()这一种。
- 算法描述：i∈[0,N-1]
  - 计算i对括号的可行序列A；
  - 计算N-i-1对括号的可行序列B；
  - 组合得到A(B)。
  - 注：加上额外一对括号()，总括号共N对

Code

```
1:  () () () () ()
2:  () () () (())
3:  () () (()) ()
4:  () () (() ())
5:  () () ((()))
6:  () (()) () ()
7:  () (()) (())
8:  () (() ()) ()
9:  () ((())) ()
10: () (() () ())
11: () (() (()))
12: () ((()) ())
13: () ((() ()))
14: () (((())))
15: (()) () () ()
16: (()) () (())
17: (()) (()) ()
18: (()) (() ())
19: (()) ((()))
20: (() ()) () ()
21: (() ()) (())
22: ((())) () ()
23: ((())) (())
24: (() () ()) ()
25: (() (())) ()
26: ((()) ()) ()
27: ((() ())) ()
28: (((()))) ()
29: (() () () ())
30: (() () (()))
31: (() (()) ())
32: (() (() ()))
33: (() ((())))
34: ((()) () ())
35: ((()) (()))
36: ((() ()) ())
37: (((())) ())
38: ((() () ()))
39: ((() (())))
40: (((()) ()))
41: (((() ())))
42: ((((()))))
```

```
void Unit(vector<string>& result,
    const vector<string>& prefix, const vector<string>& suffix)
{
    vector<string>::const_iterator ip,is;
    for(ip = prefix.begin(); ip != prefix.end(); ip++)
    {
        for(is = suffix.begin(); is != suffix.end(); is++)
        {
            result.push_back("");
            string& r = result.back();
            r += "(";
            r += *ip;
            r += ")";
            r += *is;
        }
    }
}

vector<string> AllParentheses(int n)
{
    if(n == 0)
        return vector<string>(1, "");
    if(n == 1)
        return vector<string>(1, "()");
    vector<string> prefix, suffix, result;
    for(int i = 0; i < n; i++)
    {
        prefix = AllParentheses(i);
        suffix = AllParentheses(n-i-1);
        Unit(result, prefix, suffix);
    }
    return result;
}
```

思考

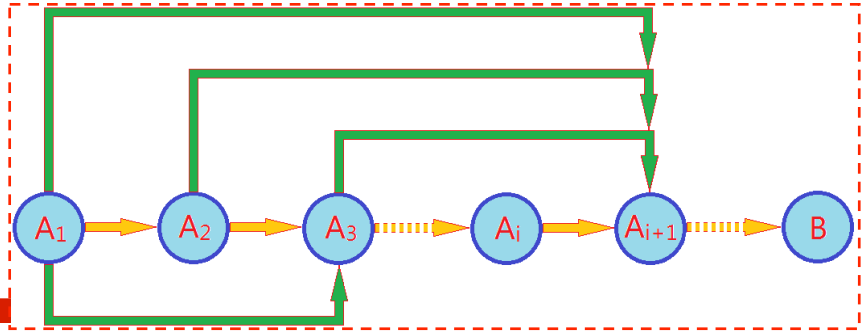
- 可以通过增加缓存的方式，对已经计算得到的字串直接获取，以空间换时间，降低时间复杂度。
- 如果只是计算可行括号串的数目，如何计算？
  - 数组A[i]表示长度为i的括号串的可行数目，即著名的Catalan数。
- Calalan数：
  - 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, 208012, 742900, 2674440, 9694845, 35357670, 129644790, 477638700, 1767263190, 6564120420, 24466267020, 91482563640, 343059613650, 1289904147324, 4861946401452……

Code

```
0: 1
1: 1
2: 2
3: 5
4: 14
5: 42
6: 132
7: 429
8: 1430
9: 4862
10: 16796
11: 58786
12: 208012
13: 742900
14: 2674440
15: 9694845
16: 35357670
17: 129644790
18: 477638700
19: 1767263190
```

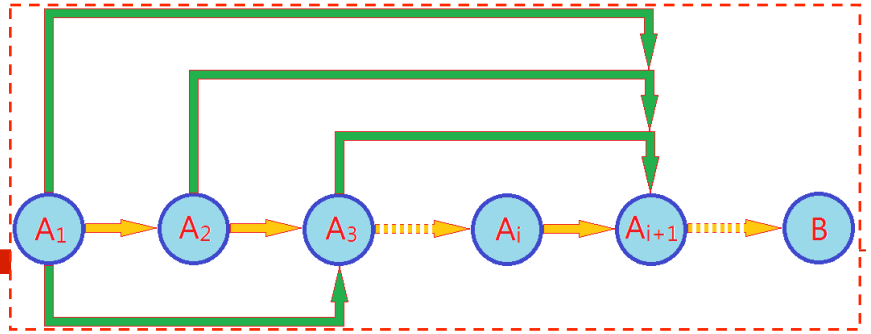
```
void GetCatalan(int* pCatalan, int N)
{
    pCatalan[0] = 1;
    pCatalan[1] = 1;
    int i,j;
    int c;
    for(i = 2; i <= N; i++)
    {
        pCatalan[i] = 1;
        c = 0;
        for(j = 0; j < i; j++)
        {
            c += pCatalan[j] * pCatalan[i-j-1];
        }
        pCatalan[i] = c;
    }
}

int _tmain(int argc, _TCHAR* argv[])
{
    const int n = 19;
    int catalan[n+1];
    GetCatalan(catalan, n);
    PrintNumber(catalan, n);
    return 0;
}
```

总结



- 动态规划是方法论，是解决一大类问题的通用思路。事实上，很多内容都可以归结为动态规划的思想。
  - KMP中求next数组：已知next[0…i-1]，求next[i]；
  - 最长回文子串Manacher算法中，已知P[0…i-1]求P[i]
- 何时可以考虑使用动态规划：
  - 初始规模下能够方便的得出结论
    - 空串、长度为0的数组、自身等
  - 能够得到问题规模增大导致的变化
    - 递推式——状态转移方程

总结



- 无后效性
  - 计算A[i]时只读取A[0…i-1]，不修改——历史
  - 计算A[i]时不需要A[i+1…n-1]的值——未来
- 在实践中往往忽略无后效性：
  - 问题本身决定了它是成立的：格子取数问题
  - 通过更改计算次序可以达到该要求：矩阵连乘问题
- 哪些题目不适合用动态规划？
  - 状态转移方程的推导，往往陷入局部而忽略全局。在重视动态规划的同时，别忘了从总体把握问题。

思考题：字符串的交替连接

- 输入三个字符串s1、s2和s3，判断第三个字符串s3是否由前两个字符串s1和s2交错而成，即不改变s1和s2中各个字符原有的相对顺序，例如当s1=“aabcc”，s2=“dbbca”，s3=“aadbbcbcac”时，则输出true，但如果s3=“accabdbbca”，则输出false。
- 换个表述：
  - s1和s2是s3的子序列，且s1∪s2=s3

Code

```
bool IsInterlace(const char* str1, const char* str2, const char* str)
{
    int M = (int)strlen(str1);
    int N = (int)strlen(str2);
    int S = (int)strlen(str);
    if(M+N != S)
        return false;
    vector<vector<bool> > p(M+1, vector<bool>(N+1));
    int i,j;
    p[0][0] = true;
    for(i = 1; i <= M; i++)      //首列
        p[i][0] = (p[i-1][0] && (str1[i-1] == str[i-1]));
    for(j = 1; j <= N; j++)      //首行
        p[0][j] = (p[0][j-1] && (str2[j-1] == str[j-1]));

    for(i = 1; i <= M; i++)
    {
        for(j = 1; j <= N; j++)
        {
            p[i][j] = (p[i-1][j] && (str[i+j-1] == str1[i-1]))
                || (p[i][j-1] && (str[i+j-1] == str2[j-1]));
        }
    }
    return p[M][N];
}
```

Code2

```
bool IsInterlace2(const char* str1, const char* str2, const char* str)
{
    int M = (int)strlen(str1);
    int N = (int)strlen(str2);
    int S = (int)strlen(str);
    if(M+N != S)
        return false;
    if(M < N)
        return IsInterlace2(str2, str1, str);
    vector<bool> p(N+1);
    int i,j;
    p[0] = true;
    for(j = 1; j <= N; j++)      //首行
        p[j] = (p[j-1] && (str2[j-1] == str[j-1]));

    for(i = 1; i <= M; i++)
    {
        p[0] = (p[0] && (str1[i-1] == str[i-1]));
        for(j = 1; j <= N; j++)
        {
            p[j] = (p[j] && (str[i+j-1] == str1[i-1]))
                || (p[j-1] && (str[i+j-1] == str2[j-1]));
        }
    }
    return p[N];
}
```

思考题：Palindrome Partitioning

- 给定一个字符串str，将str划分成若干子串，使得每一个子串都是回文串。计算str的所有可能的划分。
  - 单个字符构成的字符串，显然是回文串；所以，这个的划分一定是存在的。
- 如：s=“aab”，返回
  - “aa”，“b”；
  - “a”，“a”，“b”。

Code

```
Count: 16
1: aba | c | dccd | aa
2: a | b | a | c | dccd | aa
3: aba | c | d | cc | d | aa
4: a | b | a | c | d | cc | d | aa
5: aba | cdc | c | d | aa
6: a | b | a | cdc | c | d | aa
7: aba | c | d | c | c | d | aa
8: a | b | a | c | d | c | c | d | aa
9: aba | c | dccd | a | a
10: a | b | a | c | dccd | a | a
11: aba | c | d | cc | d | a | a
12: a | b | a | c | d | cc | d | a | a
13: aba | cdc | c | d | a | a
14: a | b | a | cdc | c | d | a | a
15: aba | c | d | c | c | d | a | a
16: a | b | a | c | d | c | c | d | a | a
```

```
void Add(vector<vector<string> >& to, const vector<vector<string> >& from, const string& sub)
{
    if(from.empty())
    {
        to.push_back(vector<string>(1, sub));
        return;
    }
    to.reserve(from.size());
    for(vector<vector<string> >::const_iterator it = from.begin(); it != from.end(); it++)
    {
        to.push_back(vector<string>());
        vector<string>& now = to.back();
        now.reserve(it->size()+1);
        for(vector<string>::const_iterator s = it->begin(); s != it->end(); s++)
            now.push_back(*s);
        now.push_back(sub);
    }
}

void MinPalindrome4(const char* str, vector<vector<string> >& all)
{
    int size = (int)strlen(str);
    vector<vector<bool> > p(size, vector<bool>(size));
    CalcSubstringPalindrome(str, size, p);

    //prefix[i]: 长度为i的前缀串的所有划分方案
    vector<vector<string> >* prefix = new vector<vector<string> >[size];
    prefix[0].clear();  //仅为了强调长度为0的串，划分解为空
    int i,j;
    for(i = 1; i <= size; i++)  //考察str[0...i-1]
    {
        for(j = 0; j < i; j++)
        {
            if(p[j][i-1])
            {
                Add((i == size) ? all : prefix[i], prefix[j], string(str+j, str+i));
            }
        }
    }
    delete[] prefix;
}
```

我们在这里

- 问题答疑：http://www.xxwenda.com/
  - 烦请邀请visio或其他人回答问题

- 本课程QQ群：
  - 471632108(小象学院：机器学习群)

感谢大家！

恳请大家批评指正！